

System Blueprint & Project Planning

1. Group Agreement and Project Plan

Studio Session No. CL07

Group No. 6

Name:

Irfan Zahidy: 105652749

Ahmed Majeed: 104822091

Mihan Hettiarachchi: 104519380

Group style and roles

The group will function collaboratively, with final decisions made as a team. We equally divide the work into three parts, and splitting it based on our strengths and experience, ensuring equal and strong contributions. As a group we allow for free communication to provide help to each other and help to push ourselves forward.

The person who will be responsible for each assessment will be delegated as a team to make sure the assessments are submitted before the deadline, based on availability and organisation. Likewise, the person who allocates themselves to ask question on behalf of the group is designated by all the team members to represent the whole group with confidence.

The scrum master will be rotated among the three group members to maintain the equal workload delegations. As such will be split into 3 sections including Problem Definition & Requirements + Data Acquisition & Preparation, in the first sprint. Model Development & Experimentation + Deployment & Integration as the second sprint. Monitoring & Maintenance + Governance & Retirement as the last sprint. Current delegations include Irfan as the scrum master in the first sprint, Ahmed in the second sprint and Mihan in the last sprint.

Sprint Plan:

Element	Details for Sprint 1	Details for Sprint 2	Details for Sprint 3
Sprint Goals	Identify vulnerable machine components and acquire data to formulate a usable dataset to train model	Train dataset extensively to determine the best classification & regression models to train the model Create a user interface and automation of the system to run reduce head costs	Consistent monitoring and intervention during an error Proper documentation and model lineage to track areas of success and failures Approval system to ensure safety
Key Deliverables	Component list with chokeholds, limitations, and history of failure Environmental risks and hazards report Necessary sensors for key failure points like heat, vibration, pressure etc. Create data set and refine missing values and outliers Establish threshold for normal to point of failure Identify key features and establish their correlation to the prevention of machine failure.	Train data in optimal split. (e.g. 70/30) Use multiple models to train the AI detection using techniques like cross validation to find the most stable and accurate model Keep the data clean to prevent AI from using patterns and ensures classes are balanced Implement docker and Kubernetes Develop real-time live updates Implement SCADA in cooperation with Modbus Develop fallback mechanism in case of system failure.	Monitorisation of model to make sure it remains sharp and consistent Implementation of data drift detection to amend thresholds based on the changes Automation of new data to retrain the model to be better incorporating physical data Proper documentation that can be traced back Limitations of the models success to be properly documented to keep workers safe Termination of old data to prevent leakage to new dataset Safety regulation compliance
Team Allocation	Project management: 100% on reports and documentation Data engineer: 100% on sensors and dataset Data scientist: 100% on features and thresholds	Project management: 100% on data organisation and data training/test split Data engineer: 100% on Model training Data scientist: 100% on Docker, Kubernetes, and SCADA implementation	Project management: 100% on safe keeping model lineage and documentation Data engineer: 100% on monitoring and maintenance Data scientist: 100% on governance and retirement
Success Criteria	Appropriate and ready to use dataset with key features and thresholds defined Stakeholders approval for the commencement of the project and agreement on the deliverables.	Develop a fully trained model that can be automated to trained with new data Create user interface to alert workers of failures	Automated monitoring of the data to prevent data drift Continue to train the model using new data Reliable model lineage and trace backs Safety compliance
Risks Addressed	Limitations of sudden events that can't be detected from ML model	Possibilities of false negatives which can lead to catastrophic results if not amended	Failure to change and plan ahead of environmental factors, machine wear etc.

Timeline:

Problem Definition & Requirements: Week 6 - 7

Data Acquisition & Preparation: Week 7 - 8

Model Development & Experimentation: Week 8 - 10

Deployment & Integration: Week 9 – 11

Monitoring & Maintenance: Week 10 - 11

Governance & Retirement: Week 12

Communication Plan

Current communication lines implemented include WhatsApp for quick messages, reminders, creating meeting times as well as the main communication platform for meetings. With meetings being facilitated by the scrum master, with another team member keeping notes from the meeting.

Documentations will proceed on google docs to allow for multiple real-time updates on the project proceedings. Likewise, the coding will be exclusively contributed to a shared GitHub folder, ensuring the work is updated with comments and history.

Meetings will be held periodically every week to keep with deadlines and check progress at a minimum. Longer meetings will be arranged prior to key deadlines to make sure all the work can be compiled and completed prior to the deadline.

Expectations for communication by all team members is that they must be available during allocated meeting times and the day of every major assignment deadline. As well as be reasonable responsive on the main communication line.

On-track behaviours

Early submission of contributions is essential in good workflow situations. As, the assessment will require certain components completion prior to the initiation of other components.

Collaboration will be crucial during this period as the workload will increase in difficulty and will require collaborative effort and attention from all team members.

Constructive feedback will be essential to overcome difficult tasks and ensure the project is heading in the right direction. Availability for meetings and proactive initiations will allow deadlines to be met successfully, and plan ahead for possible setbacks or hardware malfunctions.

Off-track behaviours

Examples of being unorganised and unco-operative, such as delaying key deadlines, preventing the advancement of the project by being lazy and distant.

Missing important meetings and leaving deadlines to the last minute

Not providing feedback or engaging in collaborative efforts, not taking initiatives to better the project outcome.

Not preparing for malfunctions or setbacks thus, being faced with them without any time to make adjustments before the final deadline.

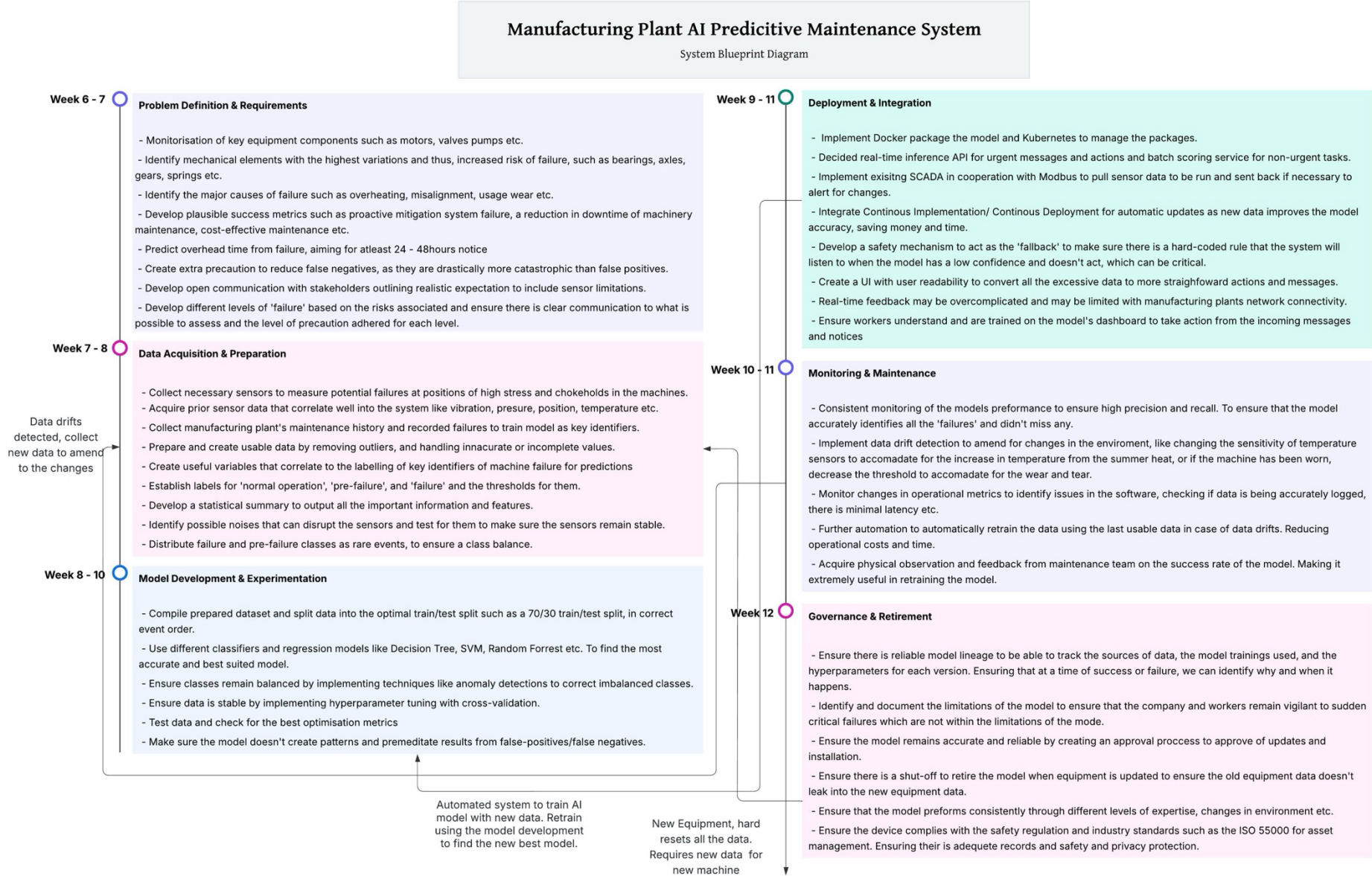
Resolving tensions

We will attempt to resolve off-track behaviour through honest and professional communication within the group to adjust for any actions. In a situation where tensions arise, the matter will be escalated through the appropriate forums and documented to the available tutor/lecturer.

Team Expectations:

1. All members will contribute equally and meet deadlines
2. Clear communication will be maintained throughout the project
3. All members will actively participate in meetings and discussion

2. System Blueprint Diagram:



3. MLOps & DevOps Integration (High-Level Design): How MLOps (retraining, monitoring) and DevOps (CI/CD, version control) will fit into your project.

Data & Training Strategy

The Dataset will be split chronologically into 70% training data and 30% test data. A chronological split is essential rather than a random split because the sensor data is collected continuously throughout the year and is subject to seasonal variation for example, temperature readings differ significantly between summer and winter. Randomly shuffling the data would cause temporal leakage, where the model learns from future data points during training, producing falsely optimistic evaluation results that do not reflect real deployment conditions. During development, 5-fold cross-validation will be applied on the training portion to tune hyperparameters and compare models without exposing the held-out test set. Additionally, as the deployed system collects new confirmed failure records from the maintenance team, this new labelled data will be incorporated into future retraining cycles to keep the model accurate over time.

Retraining Strategy

Three criteria performance-based, time-based, and drift-based will cause the model to retrain. Instead of monitoring overall accuracy, the performance-based trigger focuses on recall. This distinction is essential since a model that predicts "no failure" for every input will still achieve 96% accuracy while completely missing every actual failure in a dataset where 96.6% of readings are non-failure. Recall measures the number of real failures that the model accurately identified, which is a direct indicator of the system's usefulness. Retraining is automatically started if recall falls below 80% because, below that level, actual failures are being overlooked at a rate that results in unscheduled downtime, emergency repair expenses, and possible worker safety risks.

Regardless of performance, the time-based trigger retrains the model once a month. This accounts for two gradual changes that recall alone would not catch quickly enough: seasonal variation in sensor readings across the year, and the slow mechanical wear of industrial equipment over time. Without regular retraining, the model slowly loses accuracy before any warning threshold is reached. Machines do not decline suddenly; instead, they modify slowly month by month.

Of the three, the drift-based trigger is the most effective. It continuously analyses the statistical distribution of incoming sensor data, such as pressure readings, temperature ranges, and vibration levels, to the baseline set during training. Because drift might happen before recall drops at all, this trigger is crucial. For example, if new equipment is installed in the middle of the month, the sensor distributions change right away, although recall can seem good for a few weeks before the model starts to overlook problems. Drift detection detects this change as soon as it occurs, starting early retraining and prevents failures from going unnoticed.

Monitoring & Drift Detection

After deployment, the system will continuously monitor three categories: sensor data distributions, model performance metrics, and operational metrics. Operational metrics monitor the system's overall health, including inference delay, memory usage, uptime, and error rates, to make sure the system is operating dependably and returning predictions within a reasonable time frame for the maintenance team to act upon. Model performance metrics track recall continuously as ground truth becomes available, i.e., once the maintenance team confirms whether a predicted failure actually occurred. Recall is monitored instead of accuracy because, according to the retraining strategy, a high accuracy score may hide a model that is failing to identify actual failures.

By comparing the weekly mean and standard deviation of live received temperature, vibration, and pressure readings to the baseline statistics set during training, sensor data distributions are checked for data drift. A warning occurs and retraining begins if the difference exceeds a certain threshold, such as a mean temperature shift of 5°C. This takes into consideration real-world variations like changes in the seasons or new environmental circumstances like increased humidity or dust that the model was not trained on.

By tracking recall over time using verified results, concept drift can be detected by itself. The link between sensor readings and failure outcomes may have altered, perhaps as a result of a machine upgrade, rather than the input data itself, if recall declines without any corresponding data drift being found. In this case, the model is also matched with the current operational conditions by retraining.

DevOps Practices

GitHub will be used for version control of all code, preprocessing scripts, and model files. The issue of many team members working on different parts of the pipeline at the same time is resolved by this; in the absence of version control, pushing code changes would lead to conflicts and overwrite each other's work. GitHub handles these conflicts, keeps a complete record of all changes, and lets the team go back to any earlier code version if necessary. GitHub Actions will be used to perform automated checks each time a team member uploads code to GitHub. These tests validate that the model training script runs without error, that the preprocessing pipeline runs correctly, and that data validation passes, such as verifying that there are no missing sensor columns and that values fall within expected ranges. The modified pipeline is allowed only when every check is successful. This pipeline, known as Continuous Integration and Continuous Delivery, makes sure that no malfunctioning code gets into the system.

Every trained model will be stored as a.pkl file with a version identifier, such as model_v1_2026-04-01.pkl. This file will be followed with a model card that details the training dataset version, hyperparameter settings (such as kernel type and C value), and recall and accuracy scores attained during validation. This prevents every model version is completely repeatable and auditable by tracking what your Seminar 3 notes refer to as lineage, which includes data, code, and hyperparameters. The team can quickly roll back by redeploying the prior.pkl file and using its model card to verify that it was the better performing version if a newly retrained model performs worse than the prior version. This removes the need for memory or guessing and makes rollback safe and dependable.

4. Project Management Setup: A Jira board with key user stories and a Confluence/Notion/Word page for project documentation.

Project Management Environment

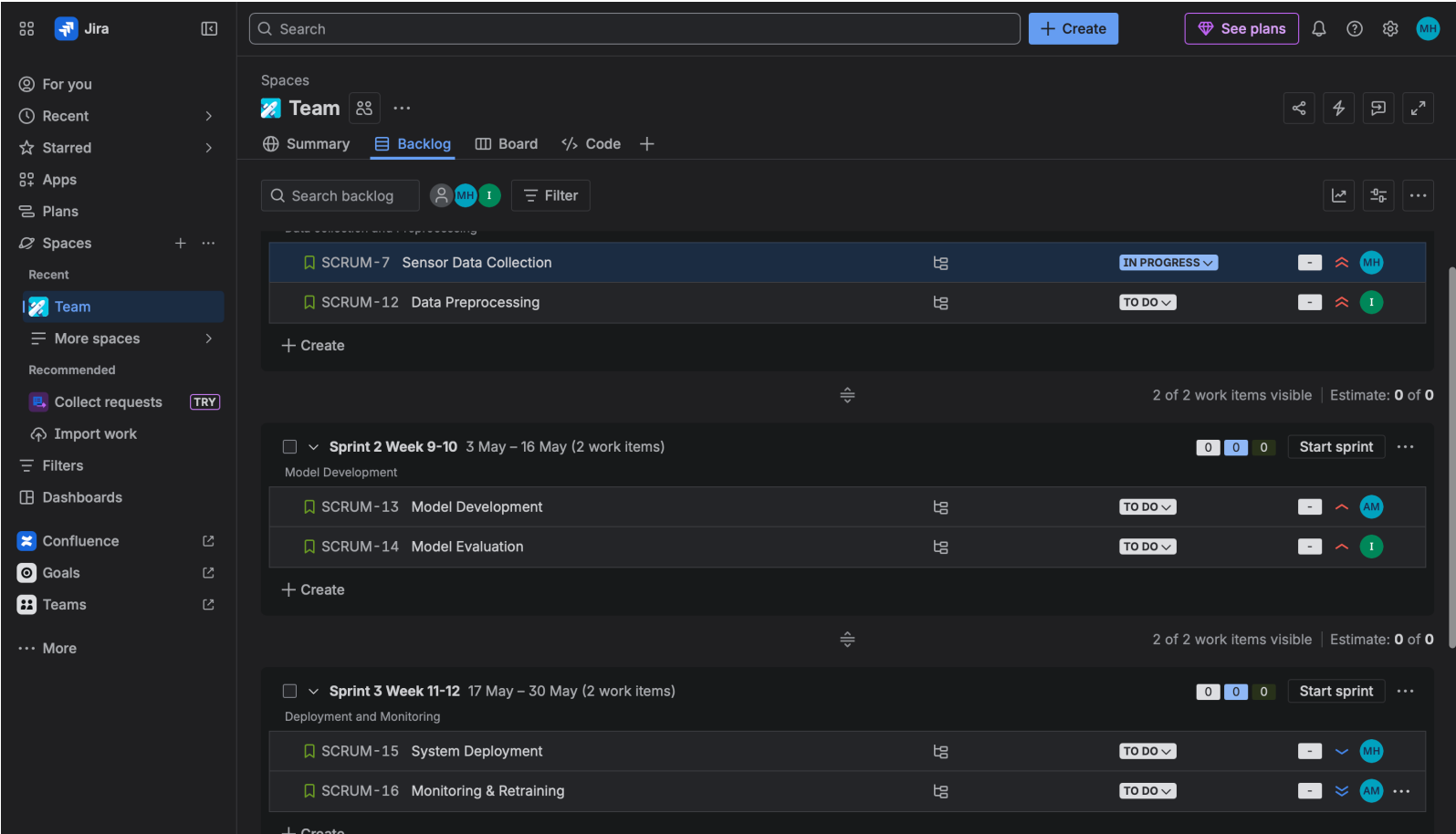


Figure 1: Jira page showing user stories organised into 3 sprints aligned with AI engineering lifecycle

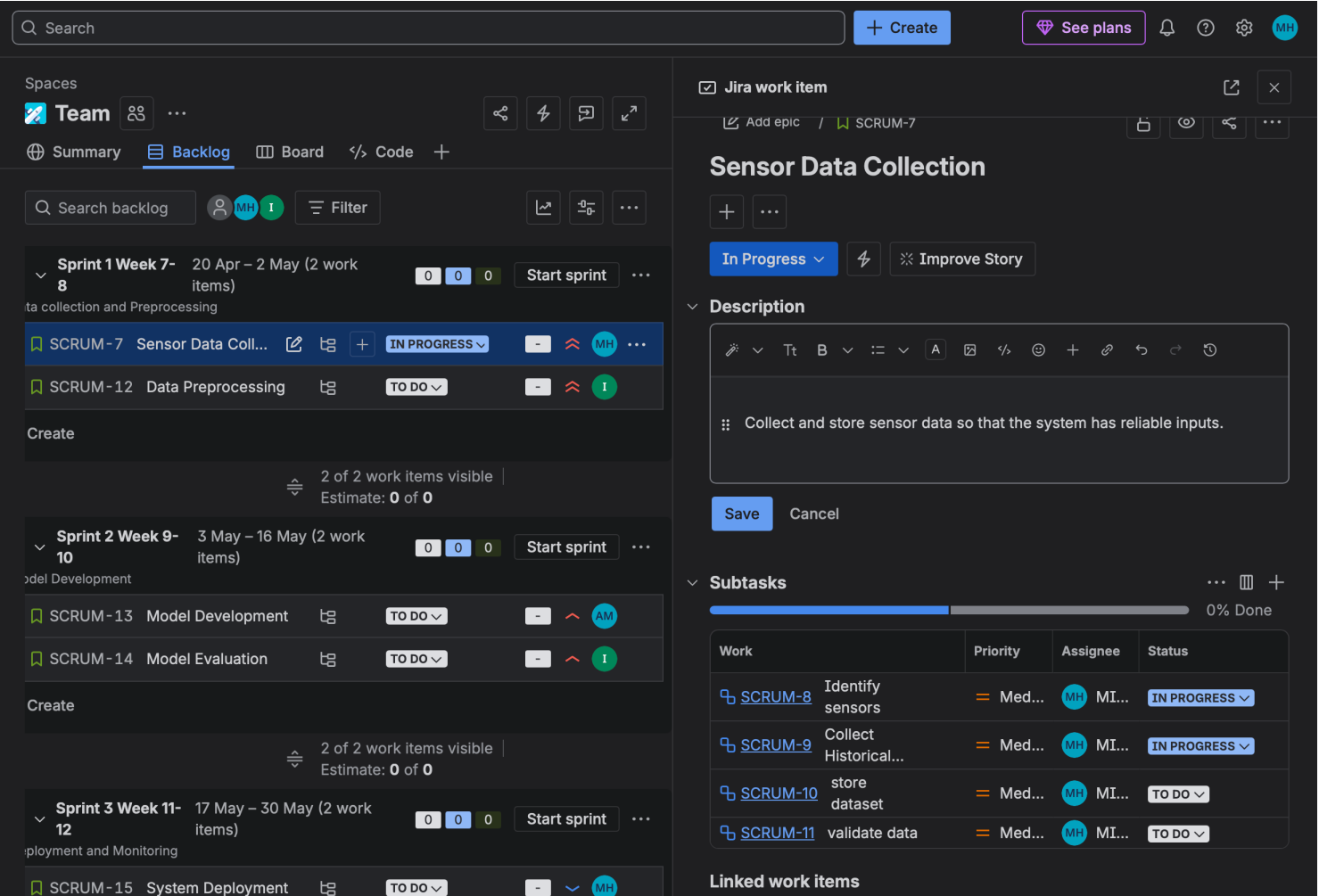


Figure 2: Sensor Data Collection user story with task breakdown into subtasks, which showcases detailed planning and task allocation.

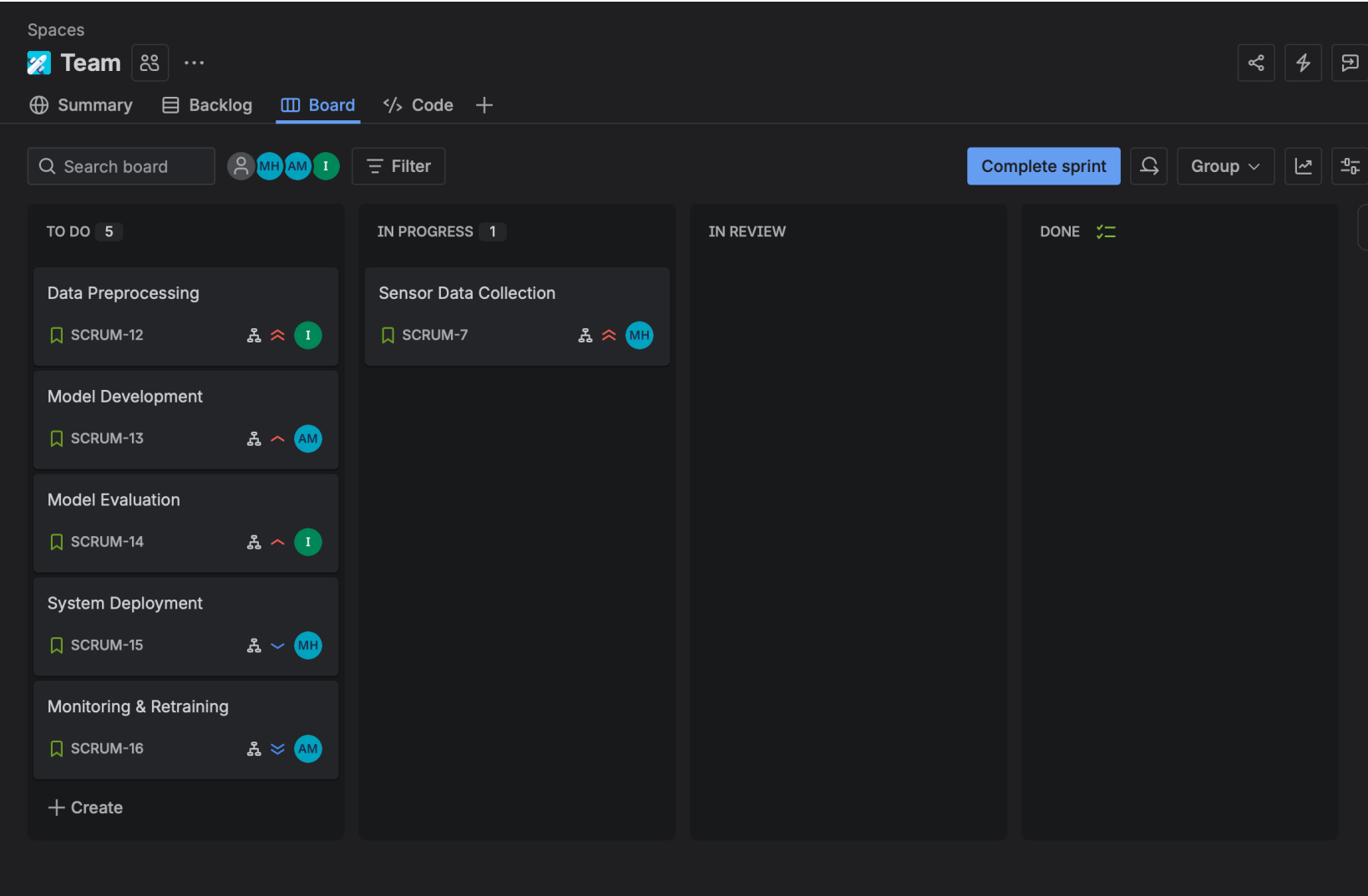


Figure 3: Jira Board view showing currently started sprints, in different workflow stages (To-Do, In-progress, In-review, Done)

We used a Jira board to implement Agile project management practices to our project. User stories were made to complete key system functionalities and were broken down into smaller subtasks to ensure clear task allocation and execution. The project was split into three sprints to fit our projects AI engineering lifecycle: data preparation, model development, and deployment and monitoring. Tasks were then given priority ratings and allocated to team members.

Shared Documentation Space

 Software Development ...

 Content + ...

Q Search by title

I •  AI Predictive Maintenance System

•  System Overview

•  Key Design Decisions

+ Create

Figure 4: Confluence documentation space showing project pages

The screenshot shows a Confluence page for a project. On the left is a sidebar with navigation options: 'For you', 'Recent', 'Starred', 'Spaces', 'Apps', 'Software Development', and 'Content'. The 'Content' section is expanded, showing a search bar and a list of items: 'AI Predictive Maintenance System' (selected), 'System Overview', and 'Key Design Decisions'. The main content area has a title 'AI Predictive Maintenance System' at the top right, with 'Published 33m ago' and user avatars. Below the title is the author 'By MIHAN HETTIARACHCHI' and an 'Add a reaction' button. The body text describes the project's goal: to develop an AI-based predictive maintenance system for manufacturing plants using sensor data (temperature, vibration, pressure) to predict equipment failures and reduce downtime, costs, and safety risks.

For you
Recent
Starred
Spaces
Apps

Software Development
Content

Search by title

- AI Predictive Maintenance System
 - System Overview
 - Key Design Decisions

Published 33m ago

AI Predictive Maintenance System

By MIHAN HETTIARACHCHI

This project aims to develop an AI-based predictive maintenance system for manufacturing plants. The system uses sensor data such as temperature, vibration, and pressure to predict potential equipment failures before they occur. This allows maintenance teams to take proactive action, reducing downtime, costs, and safety risks.

Figure 5: Confluence page for AI predictive maintenance system

For the Shared documentation we decided to use Confluence, given its ease of integration into Jira. The shared documentation space includes key project information organised into pages such as AI Predictive Maintenance System, System overview, and Key Design Decisions. This shared documentation space ensures all team members have access to up-to-date documentation throughout the project lifecycle.

Contribution Statements

I Mihan Hettiarachci (104519380), contributed to the overall development of the Project Management Setup, including the creation of the Jira board with user stories, subtasks, sprint planning, task allocation and timeline for task completion. I was also responsible for setting up the Confluence shared workspace, including the AI predictive maintenance system, system overview, and key design decisions pages. I also contributed to reviewing and refining the overall report to ensure compliance across all sections.

I, Irfan Zahidy (105652749), contributed to the group agreement and project plan, outlining a clear structure to the group dynamic, delegation of roles and equal contributions to workload. As well as maintaining standards for good project management, forms of communication and conflict resolution guidelines.

I also contributed to the system blueprint diagram, outlining all the goals, key functions and potential challenges associated with each sprint. As well as assigning reasonable deadlines for each section that is attainable to achieve to keep within the assessment guidelines. I also contributed to the structure, format and review of the final document.